CampusHUB — Services, Contracts & Schema

Assignment 2 Design Benchmark | Based on the uploaded campusEventHUB repository

Team ID: 18 **Team Leader:** Yogita Gothi (20251651107)

Members: Janvi Goad (20251651047) · Shreya Verma (20251651086) · Sonali Choudhary (20251651088)

Scope note: The provided repository is campusEventHUB, a college event, club and activity platform, not a food-ordering CampusEats app. To keep this submission consistent with the actual project, it reuses the six implemented backend service modules and specifies *registerForEvent* as the central operation rather than food-order *placeOrder*.

1. Capability List

Capability (verb)	What it does
Discover events	Browse event catalogue and view event details.
Search and filter events	Find events by keyword, category, or timing.
Authenticate users	Register accounts, log in, retrieve own profile, enforce role checks.
Manage events	Create, update, list, and delete events; maintain schedule, venue, capacity and status.
Browse and manage clubs	List club profiles and allow administrators to create, update, or remove clubs.
Register for events	Create/cancel a student registration and check its status.
Review attendees	Allow an event owner/admin to view registrations for an event.
Publish announcements	Create, update, publish, list, and delete campus-wide announcements.
Notify users	List notifications, count unread items, mark one/all as read, and send event/registration updates.
View profiles and dashboards	Present role-appropriate profile and summary information.

2. Service Boundaries & Ownership

The repository has six service modules. Each owns its domain data; other services call operations instead of reading another service's tables directly. Implementer mapping below is a proposed team allocation for the assignment.

Service	Responsible member	Data owned
Auth & Identity	Yogita Gothi	users; login/session identity; role and account status
Event Management	Janvi Goad	events; event details, venue, schedule, capacity, status
Club Management	Shreya Verma	clubs; club profile, category, lead, tags and status
Event Registration	Sonali Choudhary	event_registrations; registration lifecycle, ticket and attendee snapshot
Announcements	Shreya Verma	announcements; published notices and author reference
Notifications	Sonali Choudhary	notifications; recipient messages, read state and notification type

See services.drawio (editable source) and services.png (export) for cross-service calls. The arrows name operations; they are contract calls, not direct database access.

3. Service Contracts

Contracts describe externally callable operations and intentionally hide storage details. "Unauthorized" means the caller lacks the required role; "InvalidInput" means required fields or values are invalid.

Auth & Identity

Operation	Input	Output	Errors
registerUser	name, email, password, requestedRole	user summary, account status	EmailAlreadyExists; InvalidRole; InvalidInput
login	email, password	access token, expiry, user summary	InvalidCredentials; AccountDisabled

Operation	Input	Output	Errors
getCurrentUser	access token	user summary: id, name, email, role	Unauthenticated; AccountDisabled
getUserSummary	userId	minimal user identity and role	UserNotFound

Event Management

Operation	Input	Output	Errors
listEvents	optional query, category, date range	event summaries[]	InvalidFilter
getEventSummary	eventId	event details, owner reference, capacity and status	EventNotFound
createEvent	actorId, title, category, venue, schedule, capacity, description	created event summary	Unauthorized; InvalidInput
updateEvent	actorId, eventId, changed fields	updated event summary	EventNotFound; Unauthorized; InvalidInput
deleteEvent	actorId, eventId	deletion status	EventNotFound; Unauthorized

Club Management

Operation	Input	Output	Errors
listClubs	optional category, query	club summaries[]	InvalidFilter
getClub	clubId	club profile	ClubNotFound
createClub	actorId, club profile	created club summary	Unauthorized; DuplicateClub; InvalidInput
updateClub	actorId, clubId, changed fields	updated club summary	ClubNotFound; Unauthorized
deleteClub	actorId, clubId	deletion status	ClubNotFound; Unauthorized

Event Registration

Operation	Input	Output	Errors
registerForEvent	studentId, eventId, attendee details if needed	registration summary, ticket reference, status	EventNotFound; RegistrationClosed; EventFull; AlreadyRegistered; Unauthorized
cancelRegistration	studentId, eventId	cancellation status	RegistrationNotFound; Unauthorized; AlreadyCancelled
listAttendees	actorId, eventId, optional status	attendee summaries[]	EventNotFound; Unauthorized
getMyRegistration	studentId, eventId	registration status or not-registered	EventNotFound; Unauthorized

Announcements

Operation	Input	Output	Errors
listAnnouncements	optional page, pageSize, publishedOnly	announcement summaries[]	InvalidPagination
getAnnouncement	announcementId	announcement details	AnnouncementNotFound
createAnnouncement	actorId, title, body, category, publishFlag	created announcement summary	Unauthorized; InvalidInput
updateAnnouncement	actorId, announcementId, changed fields	updated announcement summary	AnnouncementNotFound; Unauthorized
deleteAnnouncement	actorId, announcementId	deletion status	AnnouncementNotFound; Unauthorized

Notifications

Operation	Input	Output	Errors
listMyNotifications	recipientId, optional unreadOnly, page	notification summaries[]	Unauthorized; InvalidPagination
getUnreadCount	recipientId	unreadCount integer	Unauthorized
createNotification	recipientId, title, message, type, optional eventId	notification reference	InvalidRecipient; InvalidInput
markNotificationRead	recipientId, notificationId	read status and read timestamp	NotificationNotFound; Unauthorized
markAllRead	recipientId	numberMarkedRead	Unauthorized

4. Central Operation — registerForEvent

Purpose: register an authenticated student for a campus event. This is the project-equivalent core workflow for the provided campusEventHUB repository.

Inputs

• studentId (from authenticated identity; do not trust a client-supplied identity without checking it)
• eventId
• optional attendee snapshot fields required by the event (name, email, roll number, department)

Outputs

• registration reference
• event reference and current registration status (registered)
• ticket reference when ticketing is enabled
• registeredAt timestamp

Error cases

• UNAUTHENTICATED / UNAUTHORIZED — no valid student identity or role.
• EVENT_NOT_FOUND — requested event does not exist.
• REGISTRATION_CLOSED — event status or registration window disallows registration.
• EVENT_FULL — capacity is exhausted.
• ALREADY_REGISTERED — an active registration already exists.
• INVALID_ATTENDEE_DETAILS — required data is missing or invalid.
• SERVICE_UNAVAILABLE — identity or event service cannot respond; caller may retry only when safe.

Internal details hidden from callers

1. Validate the access token and obtain the canonical student identity from Auth & Identity.
2. Ask Event Management for the event summary, status, schedule and capacity; do not query its database directly.
3. Check registration eligibility and duplicate active registration. Apply capacity reservation atomically to prevent two concurrent requests from taking the final slot.
4. Create the registration in the Event Registration service and return only its public contract fields.
5. Ask Notifications to create a confirmation notification. Notification failure should not silently duplicate a registration; use an outbox/retry mechanism or report an explicit partial-notification status.
6. Keep internal database rows, password/token internals, transaction strategy, service URLs, and exception traces out of the response.

5. Database Schema & CREATE TABLE Sketch

The ER diagram is in schema.drawio and schema.png. schema.sql gives the equivalent table definitions. PK = primary key; FK = foreign key. Columns are aligned to the repository models, with a few integrity constraints added as design recommendations.

Service owner	Table	Primary key	Foreign-key references (conceptual)
Auth & Identity	users	id	—
Event Management	events	id	owner_id → users.id
Club Management	clubs	id	—
Event Registration	event_registrations	id	event_id → events.id; student_id → users.id
Announcements	announcements	id	published_by_user_id → users.id
Notifications	notifications	id	recipient_user_id → users.id; event_id → events.id (nullable)

Boundary note: A logical FK points to another service's identifier but does not transfer ownership of the referenced data. In independently deployed services, enforce cross-service validity through API calls/events; do not rely on cross-database foreign-key constraints. The SQL sketch uses FKs to make the benchmark ER relationships explicit in a single database.

6. Service Property Validation

Property	Assessment	Fix / improvement
Reachable	Mostly yes — backend modules expose HTTP routes via FastAPI; some modules depend on a shared app/database setup.	Publish stable service endpoints and health checks; route through an API gateway if separately deployed.

Property	Assessment	Fix / improvement
Self-contained	Partial — modules are split by domain, but they share one SQLAlchemy Base/database and foreign keys across service-owned data.	Give each service ownership of its persistence and migrations; exchange IDs and summaries via contracts.
Has a contract	Yes, at API level — routes and schemas exist; this document formalizes caller-facing operations and errors.	Version OpenAPI schemas and standardize error payloads, auth rules, and pagination.
Independent	Partial — shared database and synchronous cross-module assumptions create coordinated changes.	Deploy and test services independently; avoid direct cross-service table reads and use contract tests.
Loosely coupled	Partial — registrations, events, and notifications are linked by direct orchestration and shared data identifiers.	Use narrow APIs/events, idempotent consumers, timeouts/retries, and an outbox for notifications.

Submission checklist

design.pdf · services.drawio · services.png · schema.drawio · schema.png · schema.sql

Reference basis: uploaded CampusHUB repository README.md, docs/SRS.md, and backend/services/* model/router/schema modules. Proposed service-to-member allocation is for assignment planning; adjust if the team has already agreed on different ownership.